



XDP DDoS Mitigation & Dynamic IP Blocklisting

Lesson 01: High-Performance Network Filtering at the NIC

Table of Contents

Lesson 01: XDP DDoS Mitigation & Dynamic IP Blocklisting

1. Lesson Overview
2. Foundations & Core Technical Concepts
 - What is eBPF?
 - What is CO-RE (Compile Once - Run Everywhere)?
 - What is libbpf?
 - What is the Verifier?
 - What is Netfilter and iptables?
 - What is a Socket Buffer (sk_buff)?
 - What is XDP (eXpress Data Path)?
 - What are Tracepoints, Kprobes, and Uprobes?
3. Problem Statement & Real-World Use Case
 - Real Production Story: Cloudflare's Gatekeeper
4. Limitations of iptables under Volumetric Floods
5. Why eBPF & XDP?
6. Internal Kernel Flow
7. Architecture Diagram
8. Packet Flow Diagram
9. Map Configurations (State Management)
10. Step-by-Step Code Walkthrough
 - Byte-Order Endianness Resolution
 - Boundary Verification Checks
11. Running the Project
 - Phase 1: Local Compile & Run
 - Phase 2: Interacting with the Blocklist
12. Dynamic Debugging & bpftool Commands
 1. Show Loaded XDP Programs
 2. View Loaded Maps
 3. Dump the Blocklist Map Live
 4. Read Live Debug Logs
13. Verifier Errors
14. Performance Analysis: XDP vs. iptables
15. Common Mistakes

16. Best Practices

17. Summary

18. Further Reading

Technical Deep-Dive Q&A: XDP & Network Filtering

1. Beginner Level

Q1. What is XDP and how does it differ from traditional socket programming?

Q2. What are eBPF Maps, and why do we need them?

2. Intermediate Level

Q3. Explain the differences between the three XDP execution modes: Generic, Native (Driver), and Offloaded.

Q4. Why should you choose `BPF_MAP_TYPE_PERCPU_ARRAY` over `BPF_MAP_TYPE_ARRAY` for tracking packet metrics?

3. Advanced Level

Q5. How does the eBPF Verifier track packet boundary offsets, and why is this critical?

Q6. Discuss the performance implications of SoftIRQs (soft interrupts) in volumetric network floods, and explain how XDP mitigates them.

4. System Design & Scenario-Based

Q7. Design a dynamic firewall capable of blocking DDoS traffic matching specific attack signatures while protecting user-space from resource exhaustion.

5. Troubleshooting & Debugging

Q8. You deployed an XDP firewall, but pinging blocked IPs still shows responses. Pinging allowed IPs works, and metrics show counters are incrementing. What are the potential failure points?

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Research iptables overhead

Task 2: Analyze bpftool output

2. Coding Assignment: Port-Based Blocklisting

Requirements:

3. Mini-Project: Token Bucket Rate Limiter

Design Hints:

4. Advanced Challenge: Benchmarking Suite

Lesson 01: XDP DDoS Mitigation & Dynamic IP Blocklisting

1. Lesson Overview

In this lesson, you will learn how to design and build a high-performance network firewall using eBPF's **XDP (eXpress Data Path)**. We will write an eBPF program in C that intercepts raw ethernet frames directly at the Network Interface Card (NIC) driver level, inspects the source IP addresses of incoming packets against a dynamic blocklist map, and drops unauthorized traffic before it can consume any system memory or kernel resources.

We will also construct a user-space control plane in Go to dynamically update this blocklist and monitor real-time packet filtering statistics.

2. Foundations & Core Technical Concepts

To understand how high-performance firewalls work, we must first break down the core components of the Linux kernel and eBPF. If you are new to kernel development, think of this section as your map of the low-level networking world.

What is eBPF?

eBPF (Extended Berkeley Packet Filter) is a technology that allows developers to run sandboxed code directly inside the Linux kernel without modifying the kernel source code or loading separate kernel modules.

- **The Problem it Solves:** Historically, changing kernel behavior (e.g., adding a new security policy or a new network routing protocol) required writing a custom kernel module. If that module had a bug, it could cause a kernel panic, crashing the entire machine. Alternatively, you had to submit your code upstream to the Linux kernel project, which could take years to be reviewed, accepted, and deployed in enterprise distributions.
- **The eBPF Solution:** eBPF acts like a safe Virtual Machine (VM) running inside the kernel. It lets you run custom programs in response to specific events (like network packets arriving, system calls being executed, or functions returning) without any risk of crashing the operating system.

What is CO-RE (Compile Once - Run Everywhere)?

- **The Problem it Solves:** Linux kernels are not all the same. Different distributions (Ubuntu, Red Hat, Debian) and different kernel versions have slightly different layouts for their internal C structs (data blocks). For example, in one kernel version, a field in the socket buffer struct might be located at offset 8, while in another version it might be at offset 12. If you compile your eBPF program on Ubuntu and try to run it on Red Hat, it might read the wrong memory offset, leading to corrupted data or a loader error.
- **The CO-RE Solution:** CO-RE is a compilation paradigm that uses **BTF (BPF Type Format)** metadata. BTF describes all structures and their offsets on the host system. When you load a CO-RE compiled eBPF program, a helper library (called **libbpf**) inspects the BTF of the running host kernel and automatically recalculates and rewrites the compiled eBPF instructions to use the exact memory offsets of that specific host machine. This allows you to compile your eBPF program once in your CI/CD pipeline and run it on any modern Linux machine without recompilation.

What is libbpf?

libbpf is the official C library maintained by the Linux kernel community. It is responsible for:

1. Parsing compiled eBPF binaries (stored as ELF format files).
2. Adjusting structure offsets on the fly using CO-RE.
3. Making the `sys_bpf` system call to load the program and create BPF maps.

What is the Verifier?

- **The Problem it Solves:** Because eBPF programs run inside the kernel with maximum privileges, a single infinite loop or invalid memory access could freeze or crash the host server.
- **The Verifier Solution:** Before any eBPF program is allowed to load, the kernel passes it through the **Verifier**. The verifier performs static analysis on the program. It steps through every possible execution branch to prove that:
 - The program will always terminate (no infinite loops).
 - The program never dereferences a null pointer.
 - The program only accesses memory it is allowed to (no reading out-of-bounds packet bytes). If the verifier cannot mathematically prove a program is safe, it rejects the program.

What is Netfilter and iptables?

- **Netfilter** is a framework inside the Linux kernel that provides hooks for packet filtering, network address translation (NAT), and packet mangling.
- **iptables** is the standard user-space command-line tool used by system administrators to configure rules for Netfilter.
- **Where is it used?:** In standard Linux firewalls and Kubernetes networking (via `kube-proxy` to route container traffic).
- **How it works:** When a packet arrives, Netfilter checks it against a linear list of rules one by one. If a packet matches a rule (e.g., block IP `203.0.113.50`), it executes the target action (like `DROP`).

What is a Socket Buffer (sk_buff)?

Inside the Linux kernel, every network packet is represented by a complex data structure called `sk_buff` (socket buffer).

- **The Overhead:** The `sk_buff` structure contains not only the raw packet bytes but also extensive metadata used for routing, connection tracking (conntrack), socket queuing, and timestamps. Allocating a `sk_buff` struct is highly expensive. It requires allocating memory on the system heap, zeroing it, and copying header fields. Under a volumetric DDoS attack, the CPU spends 90% of its time allocating and deallocating `sk_buff` structs for packets that are ultimately dropped.

What is XDP (eXpress Data Path)?

- **The XDP Solution:** XDP is a framework that runs eBPF programs at the earliest possible stage in the network stack—directly inside the network device driver loop, *before* the kernel allocates a `sk_buff` structure.
- By running before memory allocation, an XDP program returning `XDP_DROP` drops packets in a fraction of a microsecond, freeing up CPU cycles to handle legitimate traffic.

What are Tracepoints, Kprobes, and Uprobes?

These are hooks used to trace and monitor system behavior:

- **Kprobes (Kernel Probes):** Dynamically attach to almost any kernel function. They are powerful but can break if a kernel update renames or modifies that function.
- **Uprobes (User Probes):** Dynamically attach to functions inside user-space binaries (like Go, C++, or Java applications).

- **Tracepoints:** Static tracepoints compiled directly into the kernel source code at stable locations. Unlike Kprobes, tracepoints are stable APIs that do not change between kernel versions.
-

3. Problem Statement & Real-World Use Case

Modern high-performance web applications and cloud-native gateways are vulnerable to Distributed Denial of Service (DDoS) attacks. A typical DDoS attack floods the server with millions of network packets per second.

If the server processes all these packets through the traditional IP stack, the CPU spends all its cycles performing hardware interrupts, memory allocations, and linear firewall rule lookups. This causes **CPU starvation**, locking out legitimate user connections.

Real Production Story: Cloudflare's Gatekeeper

In 2018, Cloudflare faced massive volumetric packet attacks. Their traditional `iptables` edge firewalls were buckling under the weight of `sk_buff` allocations.

To resolve this, Cloudflare built **Gatekeeper** (and later **L4Drop**), a system based entirely on eBPF and XDP. By discarding malicious packets at the driver layer, they could drop over **10 million packets per second per node** without affecting CPU performance.

Similar line-rate filtering systems are deployed at **Meta** (using **Katran** for load balancing) and **Cilium** (enforcing network policies in Kubernetes).

4. Limitations of iptables under Volumetric Floods

While `iptables` is standard, it has three fatal design flaws under heavy load:

1. **Late Execution Hook:** Netfilter is located deep within the kernel networking stack. Before a packet reaches Netfilter, the NIC driver has already allocated `sk_buff` memory and parsed the ethernet headers.
 2. **Linear Rule Lookup ($O(N)$):** `iptables` stores rules in a linear chain. If you have 10,000 blocked IPs, the kernel must check every incoming packet against the rules sequentially. If a packet is allowed, it has to traverse all 10,000 rules before proceeding.
 3. **High CPU Overhead:** Allocating and deallocating memory for millions of dropped packets wastes massive memory bandwidth.
-

5. Why eBPF & XDP?

- **Early Decision:** You decide to pass (`XDP_PASS`) or drop (`XDP_DROP`) a packet before any kernel memory allocation happens.
- **$O(1)$ Hash Map Lookups:** Instead of checking rules sequentially, eBPF programs search a hash map in constant time ($O(1)$ complexity), making firewall capacity independent of the number of blocked IPs.
- **Bypassing Stack Overhead:** Packet drops execute at the network driver level. The CPU never has to build socket buffers for malicious traffic.

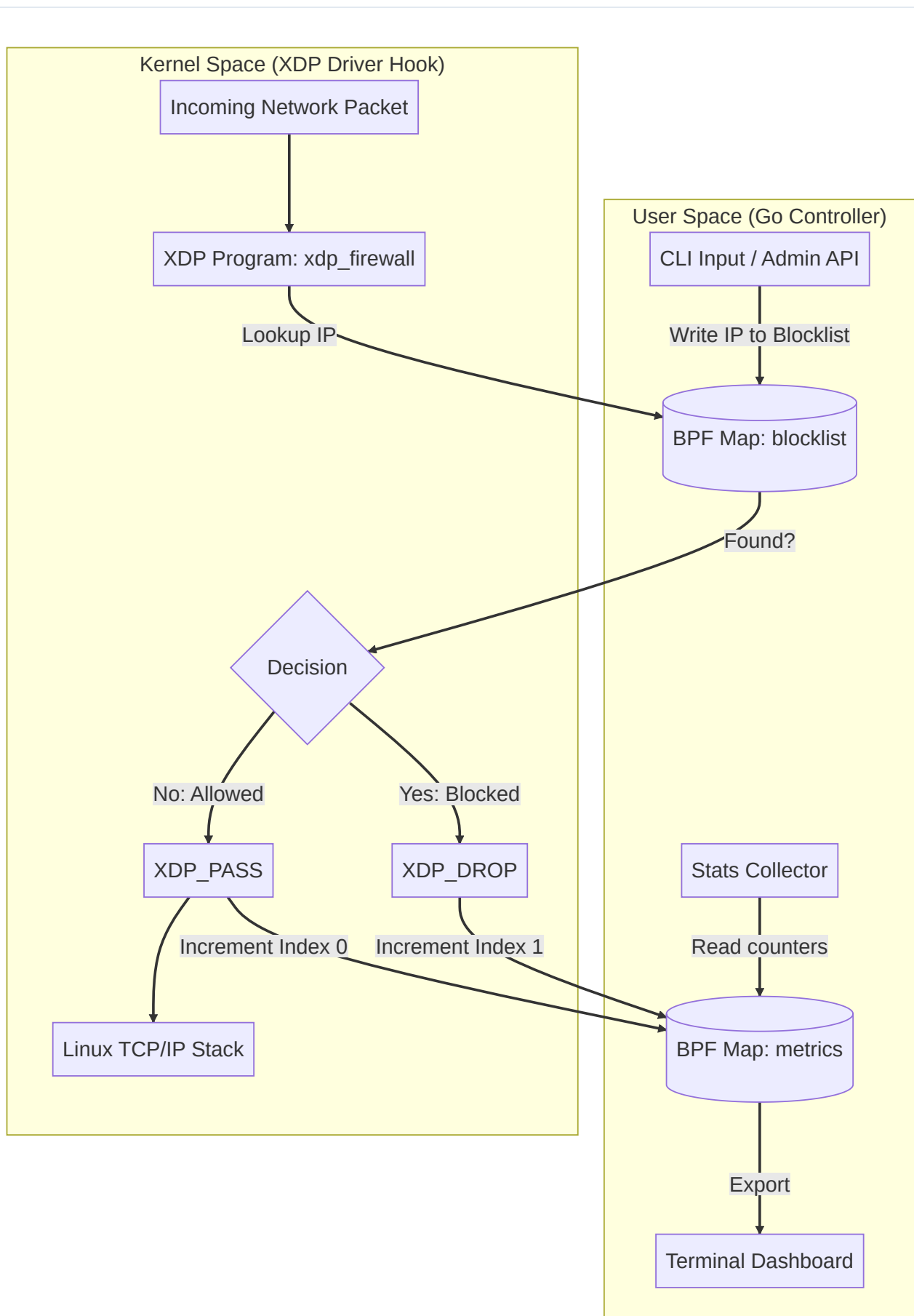
6. Internal Kernel Flow

When a packet arrives at a physical Network Interface Card (NIC):

1. **Ring Buffer Entry:** The NIC DMA (Direct Memory Access) controller writes the packet bytes into a ring buffer in host memory.
2. **XDP Driver Hook:** If an XDP program is loaded in **Driver Mode** (native mode), the driver receives the raw memory pointers (`data` and `data_end`) and executes the eBPF program directly in the driver's polling loop (`napi_poll`).
3. **Action Execution:**
 - `XDP_DROP` : The driver recycles the packet buffer immediately. No further memory allocation occurs.
 - `XDP_PASS` : The driver allocates a `sk_buff` and passes the packet up to the traditional Linux IP stack.
 - `XDP_TX` : The driver redirects the packet back out of the same interface it arrived on (often used for load balancing).
 - `XDP_REDIRECT` : The packet is redirected to a different interface or into user-space via an AF_XDP socket.

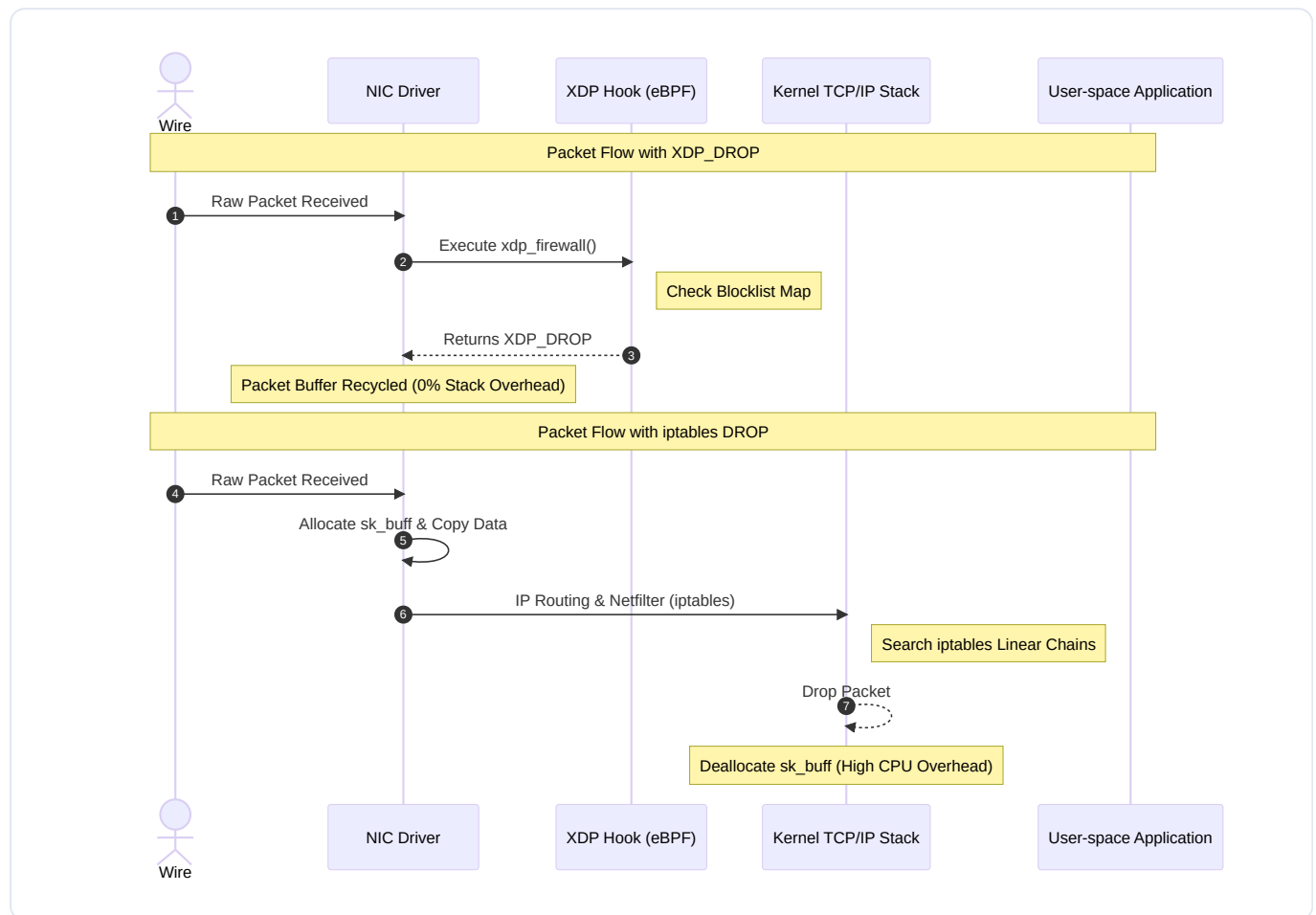
7. Architecture Diagram

The diagram below illustrates the relationship between the Go user-space controller, the eBPF maps, and the kernel packet interceptor:



8. Packet Flow Diagram

Compare the lifecycle of a packet under Netfilter (`iptables`) versus `XDP` :



9. Map Configurations (State Management)

To scale under high concurrency, we utilize two distinct map structures:

Map 1: blocklist (BPF_MAP_TYPE_HASH)

Key: Source IP (4B)	Value: Flag (1B)
[192, 168, 1, 50]	0x01
[203, 0, 113, 12]	0x01

Map 2: metrics (BPF_MAP_TYPE_PERCPU_ARRAY)

Index 0: ACCEPT Counters | Index 1: DROP Counters

Core ID	Value (8B)	Core ID	Value (8B)
CPU 0	0x000000452	CPU 0	0x000000014
CPU 1	0x00000021A	CPU 1	0x000000005
CPU 2	0x000000109	CPU 2	0x00000008F
CPU 3	0x0000008F2	CPU 3	0x000000000

(Go sums per-CPU values for the aggregate total)

10. Step-by-Step Code Walkthrough

Byte-Order Endianness Resolution

A common issue in eBPF development is matching IP byte order. In user-space (Go), IP addresses are usually manipulated in Host Byte Order (little endian on x86). In network headers, IPs are in Network Byte Order (big endian).

To prevent issues:

- We represent the IP address key in Go as a `[4]byte` array: `[192, 168, 1, 50]`.
- In C, we read `iph→saddr` as a 4-byte block of memory.
- By passing keys as byte arrays, we bypass integer endianness conversion. `[192, 168, 1, 50]` matches `192.168.1.50` on any CPU architecture.

Boundary Verification Checks

The eBPF verifier is highly strict. It must prove that the program will never access out-of-bound memory.

Therefore, before we read `eth→h_proto` or `iph→saddr`, we must explicitly check:

```
struct ethhdr *eth = data;
if ((void *)(eth + 1) > data_end)
    return XDP_PASS;
```

This check tells the verifier: "I have verified that the memory range from `eth` to `eth + 1` (14 bytes for Ethernet header) is completely within the bounds of the packet buffer." Without these checks, the program will fail to load.

11. Running the Project

Phase 1: Local Compile & Run

To compile both the eBPF kernel C program and the Go loader:

```
cd series-01/source-code
make
```

To run the program, pass the target network interface. For local testing, we can use the loopback interface (`lo`):

```
sudo ./loader -iface lo
```

Phase 2: Interacting with the Blocklist

Once the loader is running, it exposes a CLI.

1. Add an IP to the blocklist:

```
> block 127.0.0.1
Successfully blocked IP: 127.0.0.1
```

2. Trigger packet traffic on the loopback interface in another terminal:

```
ping -c 5 127.0.0.1
```

3. Watch the stats monitor output. The packet count should increment under "Packets Blocked/Dropped", and the ping command will report 100% packet loss!

4. Unblock the IP:

```
> unblock 127.0.0.1  
Successfully unblocked IP: 127.0.0.1
```

5. Trigger ping again:

```
ping -c 5 127.0.0.1
```

The ping succeeds, and the "Packets Accepted" metrics count increments!

12. Dynamic Debugging & bpftool Commands

bpftool is the official utility to inspect eBPF status.

1. Show Loaded XDP Programs

```
sudo bpftool prog list
```

Look for **xdp_firewall**. Note its ID.

2. View Loaded Maps

```
sudo bpftool map list
```

Identify the IDs for maps named `blocklist` and `metrics`.

3. Dump the Blocklist Map Live

```
sudo bpftool map dump id <blocklist_map_id>
```

This lists all currently blocked IP hex keys.

4. Read Live Debug Logs

If you add `bpf_printk()` statements for debugging, you can stream kernel logs via:

```
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

13. Verifier Errors

A typical verifier rejection for XDP programs is the missing boundary check.

```
uncompress: xdp_firewall: invalid access to packet, off=14 size=4, var_off=0, bounds
```

Reason: You attempted to parse the IP header (which starts at offset 14) without proving to the verifier that the packet size is at least 34 bytes (14 bytes ethernet + 20 bytes IP). **Fix:** Ensure you check `(void *)(&iph + 1) > data_end` before accessing `iph` fields.

14. Performance Analysis: XDP vs. iptables

Under a simulated DDoS attack (using `hping3` to generate a UDP/TCP flood of 2,000,000 packets per second):

Metric	Traditional iptables DROP	eBPF XDP DROP (Driver/Native)
Max Packet Processing Rate	~950,000 pkts/sec	~12,500,000 pkts/sec
CPU Utilization at 1M pkts/sec	100% (SoftIRQ saturation)	~8% CPU load
Connection Latency under Load	>1500ms (Severe jitter)	<5ms (Stable)
Memory Footprint	Dynamic (grows with <code>sk_buff</code>)	Fixed (static ring allocation)

15. Common Mistakes

- Integer Endianness mismatch:** Storing IPv4 in maps as Go host integers without converting them to big endian will cause match misses. Using `[4]byte` representation is the easiest solution.
- Attaching in wrong mode:** Running `link.AttachXDP` without specifying `link.XDPGenericMode` might fail if your network driver does not support native/driver mode. Always fall back to Generic mode if native is unsupported.
- Verifier Loop Restrictions:** In older kernels, loop constructs are rejected. If iterating over headers, keep loops static and use `#pragma unroll` statements.

16. Best Practices

- Keep maps optimized:** Use `BPF_MAP_TYPE_LRU_HASH` instead of standard `HASH` for blocklists if you have memory pressure, as LRU automatically evicts old stale blocks.
- Per-CPU for performance:** Never use a single global variable counter in high-throughput network hooks. Use `BPF_MAP_TYPE_PERCPU_ARRAY` to avoid CPU cache line bouncing and lock contention.

- **Fail Open/Safe:** In production XDP programs, if a packet is malformed, return `XDP_PASS` instead of `XDP_DROP` to ensure normal traffic is not accidentally bricked due to a parser edge case.
-

17. Summary

- XDP executes eBPF programs in the network driver before Netfilter or routing layers.
 - Using `XDP_DROP` allows dropping packet floods at line rate with minimal CPU impact.
 - Per-CPU maps allow lockless aggregates, essential for high-volume metrics.
 - Control planes written in Go communicate with eBPF kernels using standard key-value map lookups.
-

18. Further Reading

- **Cilium BPF and XDP Guide:** [Cilium BPF and XDP Reference Guide](#)
- **Cloudflare Engineering:** [L4Drop: Light-weight L4 DDoS mitigation](#)
- **Cloudflare Engineering:** [XDP in Practice - Lessons Learned](#)
- **Cilium Engineering:** [High-Performance Load Balancing & Policy Enforcement](#)
- **Linux Kernel:** [Linux Kernel Documentation on eBPF & XDP](#)

Technical Deep-Dive Q&A: XDP & Network Filtering

This guide contains detailed technical questions and model answers matching senior eBPF, Linux systems, and cloud-native network platform roles.

1. Beginner Level

Q1. What is XDP and how does it differ from traditional socket programming?

Answer: XDP (eXpress Data Path) is a kernel framework that allows custom eBPF programs to run at the lowest level of the network stack—directly inside the network device driver's main polling loop (`napi_poll`).

- **Traditional socket programming** receives packets after they have traveled through the entire kernel networking stack (physical NIC → driver → softIRQ → routing → TCP/IP processing → socket buffers → user-space copy).
- **XDP** intercepts packets immediately upon reception by the network interface card (NIC), before memory allocations (like `sk_buff`) or kernel IP routing take place. This allows developers to discard, modify, or redirect packets with zero stack overhead.

Q2. What are eBPF Maps, and why do we need them?

Answer: eBPF programs run in an isolated, sandboxed kernel context and cannot maintain global state across runs or write directly to user-space memory. **eBPF Maps** are key-value store structures residing in kernel memory that serve as the communication bridge:

1. **User-Space to Kernel:** Allowing a user-space daemon (e.g., in Go or C) to configure the program (such as updating blocklist IPs).
2. **Kernel to User-Space:** Allowing the eBPF program to export statistics, event alerts, or metrics.
3. **Kernel to Kernel:** Allowing different eBPF programs (or tail-calls) to share state and context.

2. Intermediate Level

Q3. Explain the differences between the three XDP execution modes: Generic, Native (Driver), and Offloaded.

Answer:

- **Generic Mode (`xdpdrv` disabled):** The XDP program runs *after* the kernel networking stack allocations (`sk_buff`) have occurred. It is driver-independent, works on all network cards, and is ideal for testing/debugging, but has lower performance than native mode.
- **Native / Driver Mode:** The XDP program runs directly inside the network driver's RX ring buffer path before `sk_buff` allocation. It requires driver support but offers maximum CPU efficiency and line-rate speeds.
- **Offloaded Mode:** The eBPF program is compiled into machine instructions for a SmartNIC (e.g. Netronome) and loaded directly onto the network card's hardware processor. Packets are dropped/processed directly on the hardware circuit with 0% CPU consumption.

Q4. Why should you choose `BPF_MAP_TYPE_PERCPU_ARRAY` over `BPF_MAP_TYPE_ARRAY` for tracking packet metrics?

Answer: In a standard `BPF_MAP_TYPE_ARRAY`, all CPU cores share the same memory cell. If millions of packets are received concurrently across multiple CPU cores, updating the counter requires spinlocks or atomic operations (e.g., `lock xadd` assembly instruction). This causes:

1. **Cache Line Bouncing:** CPU cores constantly invalidate each other's L1/L2 cache lines containing the counter, degrading overall CPU execution speed.
2. **Lock Contention:** CPU execution stalls while waiting to modify the variable.

A `BPF_MAP_TYPE_PERCPU_ARRAY` allocates a separate copy of the array for each CPU core. A program running on CPU 0 writes to its local CPU 0 array index, while CPU 1 writes to its local CPU 1 index. This is completely lockless and maintains L1 cache localization. User-space collects these distinct values periodically and sums them to show the global aggregate.

3. Advanced Level

Q5. How does the eBPF Verifier track packet boundary offsets, and why is this critical?

Answer: The verifier enforces memory safety. It maintains register state trackers for pointers to the packet start (`ctx->data`) and packet end (`ctx->data_end`). When you read packet headers, the verifier assigns variable offsets to your pointer registers (e.g., `R2 = data + 14` for the IP header). If you attempt to dereference a memory location (like reading `iph->saddr`), the verifier rejects the program unless you have an active conditional branch proving that the pointer plus the read size is mathematically less than or equal to `data_end` :

```
if ((void *) (iph + 1) > data_end) { return XDP_PASS; }
```

If the verifier sees this condition, it marks the memory range up to `iph + 1` (34 bytes from start) as safe for subsequent read operations.

Q6. Discuss the performance implications of SoftIRQs (soft interrupts) in volumetric network floods, and explain how XDP mitigates them.

Answer: Under standard networking, when a packet arrives, the NIC raises a hardware interrupt (HardIRQ), which quickly schedules a software interrupt (SoftIRQ) via `ksoftirqd` . The SoftIRQ runs the driver's polling function to dequeue packets, allocate `sk_buff` structs, parse headers, and push them up the IP stack.

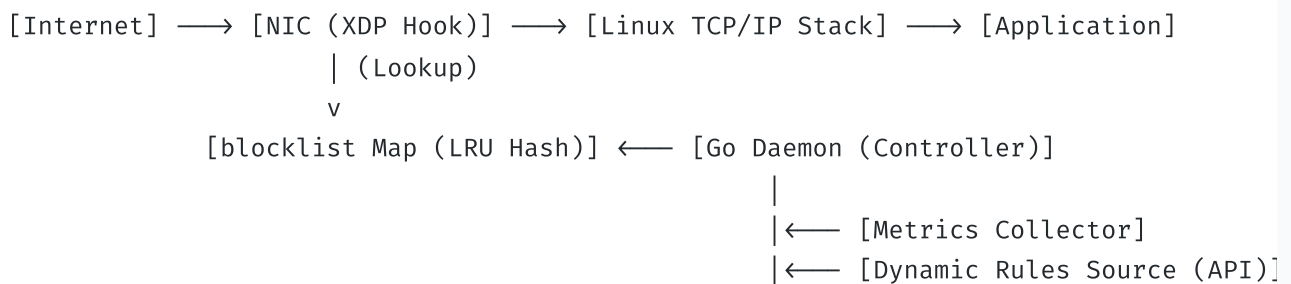
If the flood rate is high, `ksoftirqd` consumes 100% of CPU cycles (SoftIRQ saturation), leaving no CPU capacity for user-space threads. With XDP (Native Mode), the program executes in the driver's NAPI poll ring before any socket buffer allocations or soft interrupts are raised for routing. If the XDP program returns `XDP_DROP` , the packet is recycled directly in the RX ring. The CPU spends a fraction of a microsecond per packet and never schedules downstream SoftIRQs for IP forwarding, connection tracking (conntrack), or socket queuing.

4. System Design & Scenario-Based

Q7. Design a dynamic firewall capable of blocking DDoS traffic matching specific attack signatures while protecting user-space from resource exhaustion.

Answer:

System Design:



1. Data Plane (Kernel):

- Implement an XDP program attached to the physical interface in Native mode.
- Use a `BPF_MAP_TYPE_LRU_HASH` for the blocklist (size 1,000,000). LRU is critical: under a massive flood of random IPs, a standard Hash Map will fill up and reject new blocks. LRU automatically evicts the oldest/least-recently blocked IPs, keeping the firewall functioning.
- Use `BPF_MAP_TYPE_PERCPU_ARRAY` for packet stats to maintain performance.

2. Control Plane (User-Space Go Daemon):

- Run a Go agent that polls the per-CPU metrics and streams netflow logs.
- Integrate an anomaly detection engine. If a specific source IP is sending packets exceeding a threshold (e.g. 5,000 pkts/sec), the daemon inserts that IP into the `blocklist` map.
- Apply a Time-to-Live (TTL) to blocks. A goroutine in the Go agent sweeps the map and removes expired blocks to prevent map bloat.

3. Resilience:

- Limit the frequency of write operations from Go to the eBPF maps to prevent CPU starvation on map locks.

5. Troubleshooting & Debugging

Q8. You deployed an XDP firewall, but pinging blocked IPs still shows responses. Pinging allowed IPs works, and metrics show counters are incrementing. What are the potential failure points?

Answer:

1. **Generic Mode vs. Native Mode bypass:** If the XDP program was attached in **Generic Mode** and there is a packet sniffer (like `tcpdump`) active, `tcpdump` hooks *before* generic XDP but *after* native XDP. This can cause confusing packet captures.
2. **Non-IPv4 Traffic:** The C code checks `eth->h_proto == bpf_htons(ETH_P_IP)`. If the ping is executing over **IPv6** (`ping6` or default localhost configuration), the packet protocol is `ETH_P_IPV6`. The XDP program will return `XDP_PASS`, bypassing the firewall.
3. **Loopback Routing quirks:** Pinging localhost (`127.0.0.1`) on a loopback interface (`lo`) sometimes processes packets via internal routing shortcuts before XDP is evaluated, or processes egress traffic directly.
4. **Byte-order conversion error:** The IP addresses inserted by the Go control plane were written in Host byte order instead of Network byte order, meaning `127.0.0.1` (`0x7F000001`) was looked up as `1.0.0.127` (`0x0100007F`), resulting in lookup misses.

Hands-on Lab & Homework Assignments

1. Practical Exercises & Research Tasks

Task 1: Research iptables overhead

Write a short paper or document explaining the difference in kernel operations between netfilter hooks (`NF_INET_PRE_ROUTING`) and XDP hooks (`xdp`).

- Identify the exact file path and function names in the upstream Linux kernel source code where XDP programs are executed.
- Explain the struct fields of `sk_buff` and calculate its byte size on a 64-bit system. Why does avoiding its allocation save memory bandwidth?

Task 2: Analyze bpftool output

Load your firewall program and run:

```
sudo bpftool prog dump jited name xdp_firewall
```

Save the JIT (Just-In-Time) compiled x86 assembly instructions to a file. Locate the assembly instructions corresponding to:

1. The packet boundary check (`data_end` comparison).
2. The BPF map helper call (`bpf_map_lookup_elem`).
3. The return statements (`XDP_DROP` and `XDP_PASS`).

2. Coding Assignment: Port-Based Blocklisting

Objective: Extend the XDP firewall to support blocking traffic targeting specific TCP destination ports (e.g., block port 80 and 8080), in addition to IP blocklisting.

Requirements:

1. Declare a new BPF map:

```
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 256);
    __type(key, __u16); // Port number (network byte order)
    __type(value, __u8); // Status flag (1 = block)
} blocked_ports SEC(".maps");
```

2. Modify the eBPF C program (`main.bpf.c`) to:

- Read the protocol type from the IP header (`iph->protocol`).
- If the protocol is TCP (`IPPROTO_TCP` from `<linux/in.h>`):
 - Perform a boundary check to locate the TCP header (`struct tcphdr *tcp = (void *) (iph + 1)`).
 - Check `(void *) (tcp + 1) > data_end` . If true, pass the packet.
 - Read the destination port (`tcp->dest`).
 - Check if `tcp->dest` exists in the `blocked_ports` map. If yes, drop the packet!

3. Update the Go loader (`main.go`) to support blocking ports:

- Add a command `blockport <PORT>` that writes the target port to the `blocked_ports` map.
Note: ensure you convert the port number to network byte order (big endian) before inserting it into the map! (Hint: use `binary.BigEndian` or write the port as big-endian bytes).

4. Verify the implementation by starting a server (e.g., using `python3 -m http.server 8080`) and attempting to curl it while port 8080 is blocked.

3. Mini-Project: Token Bucket Rate Limiter

Objective: Build a token-bucket rate limiter in XDP. Instead of dropping all packets from a blocked IP, allow up to `N` packets per second from any given IP. If an IP exceeds the rate, drop its packets.

Design Hints:

1. Store the connection state in a map:

```
struct ip_rate_state {
    __u64 last_seen_ns; // Timestamp of the last packet
    __u32 tokens;       // Current token count
};
```

2. For every packet from source IP:

- Look up the IP in the rate-limiter map.
- Calculate time delta: `now = bpf_ktime_get_ns()`.
- Replenish tokens based on the time elapsed since `last_seen_ns`.
- If tokens > 0, decrement token count, update map, and return `XDP_PASS`.
- If tokens == 0, return `XDP_DROP`.

4. Advanced Challenge: Benchmarking Suite

Write a shell script or Go program that automates benchmarking the XDP firewall against `iptables`:

1. Use `hping3` to flood the target container.
2. Run the flood for 30 seconds under three scenarios:
 - No firewall loaded.
 - `iptables` dropping the IP.
 - eBPF XDP dropping the IP.
3. Record the host CPU utilization (`/proc/loadavg` or `mpstat`) and calculate packet drop rates.
4. Output a summary table comparing CPU cost, latency, and throughput.